

Lab 01 - Compiler Construction

Outcome of this lab

At the end of this lab, you will be able to:

- set up the basic compiler-development environment on Linux, Windows, and Mac;
- explain the four build phases of a C program: preprocessing, compilation, assembly, and linking; and
- run the commands that generate `.i`, `.s`, `.o`, and the final executable from `prog.c`.

Environment Setup for Compiler Development

Linux (Ubuntu/Debian)

GCC

- **Why we need this?** GCC compiles C programs and is the main driver we will use in this lab to run preprocessing, compilation, assembly, and linking.
- **How to install it?**

```
sudo apt update
sudo apt install gcc
```

- **How to verify it is installed properly?**

```
gcc --version
```

Flex

- **Why we need this?** Flex is a lexical-analyzer generator. Later in the course, we will use it to build scanners.
- **How to install it?**

```
sudo apt install flex
```

- **How to verify it is installed properly?**

```
flex --version
```

Bison

- **Why we need this?** Bison is a parser generator. Later in the course, we will use it to build parsers.
- **How to install it?**

```
sudo apt install bison
```

- **How to verify it is installed properly?**

```
bison --version
```

GNU Make

- **Why we need this?** Make automates build steps. It becomes very useful when our compiler project grows.
- **How to install it?**

```
sudo apt install make
```

- **How to verify it is installed properly?**

```
make --version
```

VS Code

- **Why we need this?** VS Code is a convenient editor for writing C, Flex, and Bison files.
- **How to install it?**
 - **Ubuntu:**

```
sudo snap install code --classic
```

- **Ubuntu/Debian alternative:** download the official .deb package from the VS Code website and install it with:

```
sudo apt install ./code_*.deb
```

- **How to verify it is installed properly?**

```
code --version
```

GDB

- **Why we need this?** GDB is a debugger. It helps us inspect program execution and find bugs.
- **How to install it?**

```
sudo apt install gdb
```

- **How to verify it is installed properly?**

```
gdb --version
```

Git

- **Why we need this?** Git tracks changes in our code and helps us manage project versions.
- **How to install it?**

```
sudo apt install git
```

- **How to verify it is installed properly?**

```
git --version
```

Graphviz

- **Why we need this?** Graphviz helps us draw graphs such as parse trees, ASTs, and automata diagrams.
- **How to install it?**

```
sudo apt install graphviz
```

- **How to verify it is installed properly?**

```
dot -V
```

Windows

Use the **MSYS2 UCRT64** terminal for the command-line work in this course.

MSYS2

- **Why we need this?** MSYS2 gives Windows a Unix-like development environment. It is the easiest way to install GCC, Flex, Bison, Make, and GDB on Windows.
- **How to install it?**
 1. Download the installer from the official MSYS2 website.
 2. Install it in the default location: C:\msys64.
 3. Open **MSYS2 UCRT64** from the Start menu.
 4. Update the package database and core packages:

```
pacman -Syu
```

Close the terminal if instructed, reopen **MSYS2 UCRT64**, and run:

```
pacman -Su
```

- **How to verify it is installed properly?**

```
echo $MSYSTEM  
pacman --version
```

GCC

- **Why we need this?** GCC is the compiler driver we will use in all four phases.
- **How to install it?**

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

- **How to verify it is installed properly?**

```
gcc --version  
which gcc
```

Flex

- **Why we need this?** Flex will be used later for lexical analysis.
- **How to install it?**

```
pacman -S --needed flex
```

- **How to verify it is installed properly?**

```
flex --version
```

Bison

- **Why we need this?** Bison will be used later for syntax analysis.
- **How to install it?**

```
pacman -S --needed bison
```

- **How to verify it is installed properly?**

```
bison --version
```

GNU Make

- **Why we need this?** Make automates multi-file builds.
- **How to install it?**

```
pacman -S --needed make
```

- **How to verify it is installed properly?**

```
make --version
```

VS Code

- **Why we need this?** VS Code is a convenient editor for this course.
- **How to install it?**

```
winget install Microsoft.VisualStudioCode
```

- **How to verify it is installed properly?**

```
code --version
```

GDB

- **Why we need this?** GDB helps us debug programs.
- **How to install it?**

```
pacman -S --needed mingw-w64-ucrt-x86_64-gdb
```

- **How to verify it is installed properly?**

```
gdb --version
```

Git

- **Why we need this?** Git helps us keep track of our compiler project.
- **How to install it?**

```
pacman -S --needed git
```

- **How to verify it is installed properly?**

```
git --version
```

Graphviz

- **Why we need this?** Graphviz helps us visualize graphs and trees.
- **How to install it?**

```
pacman -S --needed mingw-w64-ucrt-x86_64-graphviz
```

- **How to verify it is installed properly?**

```
dot -V
```

Mac (Apple Silicon)

XCode

- **Why we need this?** Xcode Command Line Tools provide the base developer toolchain, headers, and LLDB.
- **How to install it?**

```
xcode-select --install
```

- **How to verify it is installed properly?**

```
xcode-select -p
```

Homebrew

- **Why we need this?** Homebrew is the package manager we will use to install GCC, Flex, Bison, and other tools.
- **How to install it?**

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

After installation, add Homebrew to your shell as instructed by the installer. On Apple Silicon, it is usually:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile  
eval "$(/opt/homebrew/bin/brew shellenv)"
```

- **How to verify it is installed properly?**

```
brew --version
```

GCC

- **Why we need this?** We want to use gcc consistently in this lab manual across Linux, Windows, and Mac.
- **How to install it?**

```
brew install gcc
```

Homebrew installs a versioned GCC executable such as gcc-15. To use the common command name gcc in this lab manual, add an alias to your shell:

```
echo 'alias gcc=gcc-15' >> ~/.zshrc  
source ~/.zshrc
```

Replace gcc-15 with the version actually installed on your Mac.

- **How to verify it is installed properly?**

```
gcc --version
```

Flex

- **Why we need this?** Flex will be used later for lexical analysis.
- **How to install it?**

```
brew install flex
```

Add it to your shell path:

```
echo 'export PATH="/opt/homebrew/opt/flex/bin:$PATH"' >> ~/.zshrc  
source ~/.zshrc
```

- **How to verify it is installed properly?**

```
flex --version
```

Bison

- **Why we need this?** Bison will be used later for syntax analysis.
- **How to install it?**

```
brew install bison
```

Add it to your shell path:

```
echo 'export PATH="/opt/homebrew/opt/bison/bin:$PATH"' >> ~/.zshrc  
source ~/.zshrc
```

- **How to verify it is installed properly?**

```
bison --version
```

GNU Make

- **Why we need this?** Make automates multi-file builds.
- **How to install it?**

```
brew install make
```

Homebrew installs GNU Make as gmake.

- **How to verify it is installed properly?**

```
gmake --version
```

VS Code

- **Why we need this?** VS Code is a convenient editor for this course.
- **How to install it?**

```
brew install --cask visual-studio-code
```

- **How to verify it is installed properly?**

```
code --version
```

LLDB

- **Why we need this?** LLDB is the standard debugger on macOS.
- **How to install it?** LLDB comes with Xcode Command Line Tools.
- **How to verify it is installed properly?**

```
lldb --version
```

Git

- **Why we need this?** Git helps us manage versions of our work.
- **How to install it?**

```
brew install git
```

- How to verify it is installed properly?

```
git --version
```

Graphviz

- **Why we need this?** Graphviz helps us draw parse trees, ASTs, and automata graphs.
- **How to install it?**

```
brew install graphviz
```

- How to verify it is installed properly?

```
dot -V
```

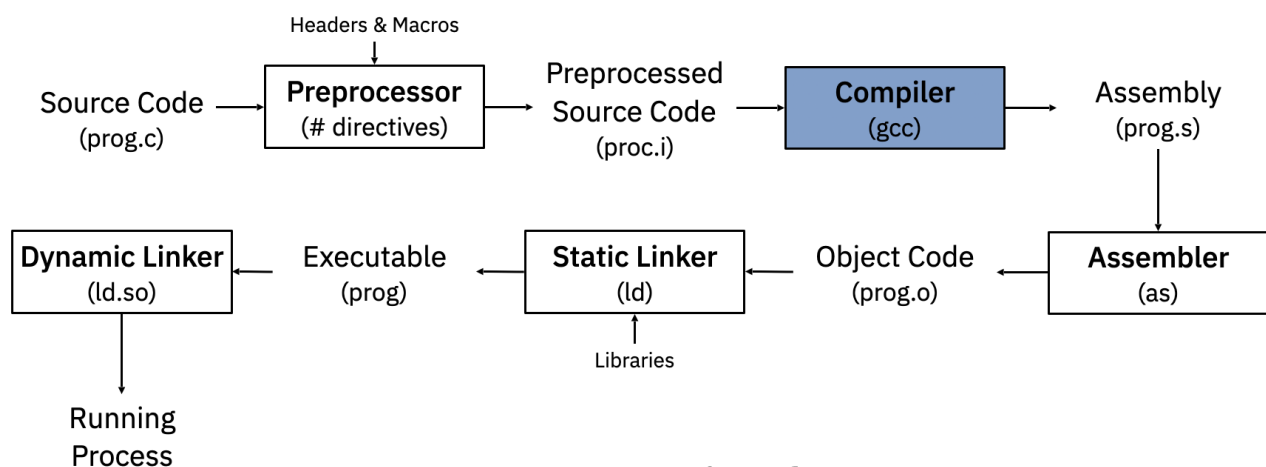
The Four Phases

Before running the commands below, create a file named `prog.c` with the following contents:

```
#include <stdio.h>
#define MESSAGE "Hello, Compiler Lab!\n"

int main(void)
{
    printf(MESSAGE);
    return 0;
}
```

The figure below shows the flow from `prog.c` to the final executable. The course focus in this lab is the four build phases: preprocessing, compilation, assembly, and linking.



From Source to Execution of a C Program

Preprocessing

Linux

- Run:

```
gcc -E prog.c -o prog.i
```

- View Output:

```
less prog.i
```

Windows

- Run:

```
gcc -E prog.c -o prog.i
```

- View Output:

```
less prog.i
```

Mac

- Run:

```
gcc -E prog.c -o prog.i
```

- View Output:

```
less prog.i
```

Compilation

Linux

- Run:

```
gcc -S prog.i -o prog.s
```

- View Output:

```
less prog.s
```

Windows

- Run:

```
gcc -S prog.i -o prog.s
```

- **View Output:**

```
less prog.s
```

Mac

- **Run:**

```
gcc -S prog.i -o prog.s
```

- **View Output:**

```
less prog.s
```

Assembly

Linux

- **Run:**

```
gcc -c prog.s -o prog.o
```

- **View Output:**

```
objdump -d prog.o
```

Windows

- **Run:**

```
gcc -c prog.s -o prog.o
```

- **View Output:**

```
objdump -d prog.o
```

Mac

- **Run:**

```
gcc -c prog.s -o prog.o
```

- **View Output:**

```
otool -tv prog.o
```

Linking

Linux

- **Run:**

```
gcc prog.o -o prog
```

- **View Output:**

```
./prog
```

Windows

- **Run:**

```
gcc prog.o -o prog.exe
```

- **View Output:**

```
./prog.exe
```

Mac

- **Run:**

```
gcc prog.o -o prog
```

- **View Output:**

```
./prog
```